Bulut Bilişim Ve Uygulamaları

BLM(3522)

ENDÜSTRİYEL IOT KARGO DRONE TAKİP SİSTEMİ- PROJE GELİŞTİRME DOKÜMANTASYONU

Hazırlayan: Berkay Bozlar

Numara: 23290939

Fakülte: Mühendislik Fakültesi

Bölüm: Bilgisayar Mühendisliği

GitHub:(https://github.com/BerkayBozlar/DroneTrackingAPI)

Tanıtım Videosu: (https://drive.google.com/file/d/15jDabFmxR5Php1RFCO4Ec85yVUppTG6M/view?usp=drive_link)

BÖLÜM 1: BACKEND (SUNUCU TARAFI) VE SİMÜLASYON ALTYAPISININ KURULMASI

Projenin temel amacı, endüstriyel kargo dronelarından gelen anlık telemetri verilerini eş zamanlı olarak işleyebilecek bir IoT altyapısı kurmaktır. Bu doğrultuda, sürekli veri üreten bir simülasyon motoru ve bu verileri anlık olarak istemciye (Frontend) iletecek WebSocket tabanlı bir köprü oluşturmak amacıyla Visual Studio ortamında ASP.NET Core projemizi başlattık.

1. Projenin Başlatılması

- **Kullanılan İDE:** Visual Studio 2022 & Visual Studio Code
- **Proje Şablonu:** ASP.NET Core Web API
- **Proje Adı:** DroneTrackingAPI
- **Çözüm Adı:** DroneTracking
- **Hedef Çerçeve:** .NET 8.0 (Uzun Süreli Destek - LTS)

Klasik Request-Response (İstek-Cevap) mimarisinin ötesine geçerek "Gerçek Zamanlı (Real-Time)" bir veri akışı kuracağımız için projeyi başlatırken minimal API mimarisi yerine, arka plan servislerini ve Hub sınıflarını daha rahat yönetebileceğim Klasör tabanlı standart yapıyı tercih ettim.

Not: Projenin bu ilk aşamasında, veri üretimi ve gerçek zamanlı aktarım için .NET 8'in yerleşik (built-in) BackgroundService ve SignalR kütüphaneleri kullanıldığından harici bir NuGet paketi indirilmesine gerek duyulmamıştır. Veritabanı ve Bulut (AWS) paketleri projenin ilerleyen fazlarında sisteme dahil edilecektir.

2. Veri Modelinin (Entity) Oluşturulması

Kargo dronelarından gelecek olan telemetri verilerinin sistemde hangi formatta ve hangi özelliklerle tutulacağını belirlemek amacıyla model sınıfımı oluşturdum.

- **Dizin:** DroneTrackingAPI/Models/
- **Oluşturulan Dosya:** DroneTelemetry.cs
- **İçerik:** Sınıf içerisine droneları benzersiz kılan DroneId, konum bilgileri için Latitude (Enlem), Longitude (Boylam), Altitude (İrtifa), enerji takibi için BatteryLevel, anlık durum takibi için Status (Örn: Görevde, Dönüşe Geçti) ve verinin üretildiği zamanı damgalamak için Timestamp özelliklerini tanımladım.

3. Gerçek Zamanlı İletişim Köprüsünün (SignalR Hub) Kurulması

Backend'de üretilen veya alınan sensör verilerinin, istemcinin (tarayıcının) sayfayı yenilemesine gerek kalmadan anında ekrana yansıtılabilmesi için Microsoft'un WebSocket kütüphanesi olan SignalR altyapısını kurdum.

- **Dizin:** DroneTrackingAPI/Hubs/
- **Oluşturulan Dosya:** DroneHub.cs
- **İçerik:** Hub sınıfından miras alan bu yapı, istemciler (React arayüzü) ile sunucu arasındaki kesintisiz çift yönlü bağlantı tünelini temsil edecek şekilde programlandı.

4. IoT Simülasyon Motorunun (BackgroundService) Yazılması

Gerçek droneların uçuş verilerini taklit edecek, 7/24 durmadan çalışacak ve sürekli koordinat/batarya verisi üretecek olan arka plan işçisini (worker) kodladım.

- **Dizin:** DroneTrackingAPI/Services/
- **Oluşturulan Dosya:** DroneSimulationService.cs
- **İçerik:** BackgroundService sınıfından miras alınarak ExecuteAsync metodu ezildi (override).
    - Sınıfın kurucu metoduna (Constructor), verileri yayınlayabilmesi için IHubContext<DroneHub> arayüzü enjekte edildi (Dependency Injection).
    - **İş Mantığı:** Sonsuz bir while döngüsü kurularak, sistemde tanımlı droneların verileri (Enlem, Boylam, İrtifa, Batarya) her saniye (Task.Delay(1000)) güncellendi ve güncellenen bu veriler SignalR üzerinden "ReceiveDroneData" etiketiyle tarayıcılara fırlatıldı.

5. Konfigürasyon ve Servislerin Tanıtılması (Program.cs)

Oluşturduğum SignalR Hub'ının ve Arka Plan Simülasyon Servisinin .NET motoru tarafından tanınması ve çalıştırılması için Program.cs dosyasında gerekli yapılandırmaları sağladım.

- **CORS Politikası:** İleride farklı bir portta (Vite - Port 5173 vb.) çalışacak olan React arayüzünün güvenlik duvarına takılmadan SignalR sunucusuna bağlanabilmesi için genişletilmiş bir CORS

politikası (AllowAnyOrigin, AllowAnyHeader, AllowAnyMethod) ekledim.

- **Servis Kayıtları:** * builder.Services.AddSignalR(); komutu ile WebSocket altyapısı sisteme tanıtıldı.
    - builder.Services.AddHostedService<DroneSimulationService>(); komutu ile yazdığım simülasyon motorunun, sunucu ayağa kalktığı anda otomatik olarak arka planda çalışmaya başlaması sağlandı.
- **Uç Nokta (Endpoint) Ataması:** app.MapHub<DroneHub>("/droneHub"); komutu ile dış dünyadan gelen websocket bağlantı isteklerinin /droneHub adresi üzerinden karşılanacağı belirtildi.

6. Yerel Test ve Geliştirme Ortamı Değişimi

Yazılan kodların doğruluğunu teyit etmek amacıyla Visual Studio üzerinden projeyi (F5 ile) başlattım. Terminal ekranında arka plan servisinin sorunsuz ayağa kalktığına dair log kayıtlarını gözlemledim.

Sunucu tarafı veri üretmeye ve bu verileri WebSocket üzerinden yayınlamaya hazır hale geldikten sonra, bu verileri görselleştirecek olan dinamik "Kontrol Paneli (Dashboard)" arayüzünü kodlamak üzere Visual Studio'yu kapatarak, daha hafif ve terminal süreçlerini yönetmesi kolay olan **Visual Studio Code (VS Code)** ortamına geçiş yaptım.

BÖLÜM 2: FRONTEND (ÖN YÜZ) GELİŞTİRME SÜRECİ VE GERÇEK ZAMANLI DASHBOARD TASARIMI

Backend tarafında oluşturulan veri akışını son kullanıcıya profesyonel, anlaşılır ve anlık olarak sunabilmek amacıyla bir "Endüstriyel Kontrol Paneli (Dashboard)" tasarlanmasına karar verilmiştir. Terminal süreçlerini ve modern web teknolojilerini daha esnek yönetebilmek adına Visual Studio Code (VS Code) ortamına geçiş yapılarak ön yüz geliştirme süreçlerine başlanmıştır.

1. React Projesinin Başlatılması ve Paket Kurulumları

Hızlı derleme (build) süreleri ve modern geliştirme deneyimi sunması sebebiyle React altyapısı, Vite derleyicisi ile birlikte kurulmuştur.

- **Terminal Komutu:** Proje ana dizininde npm create vite@latest drone-frontend -- --template react komutu çalıştırılarak ön yüz iskeleti oluşturulmuştur.
- **Dizin Değişimi ve Kurulum:** cd drone-frontend komutuyla klasöre girilmiş ve npm install ile temel Node.js paketleri indirilmiştir.
- **SignalR İstemci Paketi:** React arayüzünün, C# backend üzerinde açtığımız WebSocket tüneline (Hub) bağlanabilmesi için resmi Microsoft kütüphanesi olan SignalR paketi projeye dahil edilmiştir:

npm install @microsoft/signalr

2. Durum Yönetimi (State Management) ve SignalR Entegrasyonu

Projenin görsel mantığını ve backend ile haberleşme köprüsünü doğrudan src/App.jsx dosyası içerisinde inşa ettim. Bu dosyada gerçek zamanlı veri akışı için şu mimariyi kurdum:

- **Durum Yönetimi (useState):** Ekranda gösterilecek olan drone verilerini anlık olarak hafızada tutmak için React'in useState kancası (hook) kullanılarak boş bir dizi ([]) tanımlanmıştır.
- **WebSocket Bağlantısı (useEffect & HubConnectionBuilder):** Sayfa ilk yüklendiğinde bağlantının sadece bir kere kurulması için useEffect kancası kullanılmıştır. İçerisinde @microsoft/signalr kütüphanesinin HubConnectionBuilder metodu çağrılarak Backend adresine (http://localhost:5000/droneHub) asenkron bir tünel açılmıştır.
- **Veri Dinleme (connection.on):** Backend'in yayınladığı "ReceiveDroneData" etiketli veriler dinlenmeye başlanmış; yeni bir drone verisi (Enlem, Boylam, Batarya vb.) geldiğinde, mevcut state (durum) dizisindeki ilgili drone bulunarak sadece onun verileri güncellenmiş ve ekranın tamamı yerine sadece değişen parçaların (Virtual DOM) render edilmesi sağlanmıştır.

3. Kullanıcı Arayüzü (UI) ve Dinamik Veri Listeleme

Gelen ham telemetri verilerini okunabilir kılmak için endüstriyel standartlara uygun, Dark Mode (Karanlık Tema) tabanlı bir arayüz kodlanmıştır.

- **Dinamik Döngü (map):** State içerisinde güncellenen drone dizisi, JSX yapısı içerisinde JavaScript'in map() fonksiyonuyla döngüye

sokularak her bir drone için bağımsız bir "Drone Kartı" oluşturulmuştur.

- **Veri Bağlama:** Döngüdeki her karta drone.droneId anahtarı atanmış; Enlem (drone.latitude), Boylam (drone.longitude), İrtifa (drone.altitude) ve Durum (drone.status) gibi veriler tipografik olarak HTML etiketlerinin içine yerleştirilmiştir.
- **Görsel İndikatörler (Batarya Barı):** Gelen drone.batteryLevel verisi, CSS genişlik (width) özelliğiyle dinamik olarak eşleştirilmiş ve yüzdesel olarak azalan, görsel bir batarya ilerleme çubuğu (progress bar) tasarlanmıştır.

4. CSS Grid Mimarisi ile Kusursuz Hizalama

Altı adet kargo drone'unun ekranı taşırmadan, simetrik ve profesyonel bir düzende durması için src/App.css dosyası içerisinde modern CSS teknikleri kullanılmıştır.

- **Kapsayıcı (Container) Yapılandırması:** Ekran genişliği optimize edilerek max-width: 1400px ve margin: 0 auto özellikleri ile tüm arayüz sayfanın tam ortasına hizalanmıştır.
- **Izgara Sistemi (CSS Grid):** .drone-grid sınıfı oluşturularak display: grid tanımlanmış ve grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)) kuralı yazılmıştır. Bu sayede kartlara ideal bir boyut (min 320px) verilerek, yan yana 3 adet sığacak ve ekran küçüldüğünde otomatik alt satıra geçecek (Responsive) bir yapı kurgulanmıştır.
- **Kart Tasarımı:** Her bir drone kartına lineer renk geçişleri (linear-gradient), oval köşeler (border-radius) ve gölgelendirmeler eklenerek modern bir Kontrol Merkezi (Dashboard) görünümü elde edilmiştir.

5. Yerel Ortam (Localhost) Testi ve Doğrulama

Kodlama süreçleri tamamlandıktan sonra VS Code terminali üzerinden arayüz ayağa kaldırılmıştır:

npm run dev

Uygulama tarayıcıda açılmış, arka planda çalışan C# BackgroundService motorundan fırlatılan koordinat ve batarya değişikliklerinin milisaniyeler içerisinde React ekranına yansıdığı, State Management ve CSS Grid mimarisinin kayıpsız çalıştığı başarıyla doğrulanmıştır. Projenin yerel ayağı bu şekilde tamamlanarak, Bulut Bilişim (Cloud) ve Kalıcı Depolama (Database) entegrasyonlarına hazır hale getirilmiştir.

BÖLÜM 3: KALICI VERİ DEPOLAMA (MONGODB) VE BULUT BİLİŞİM (AWS KINESIS) ENTEGRASYONU

Projenin son aşamasında, anlık olarak üretilen ve yayınlanan telemetri verilerinin hem yerel bir veritabanında kalıcı olarak arşivlenmesi hem de bulut tabanlı büyük veri platformlarına aktarılması hedeflenmiştir. Bu doğrultuda NoSQL mimarisi ve AWS veri akış servisleri sisteme entegre edilmiştir.

1. NoSQL Veritabanı Yapılandırması (MongoDB)

Proje isterlerinde belirtilen "Gerçek zamanlı veriye uygun veritabanı" kuralını sağlamak amacıyla, JSON tabanlı doküman depolama yeteneği ve yüksek yazma hızıyla bilinen **MongoDB** tercih edilmiştir.

- **NuGet Paketi:** C# projesine MongoDB.Driver kütüphanesi dahil edilmiştir.
- **Model Revizyonu (BsonId):** Veritabanı motorunun her kaydı benzersiz bir kimlikle tanımlayabilmesi için DroneTelemetry.cs modeline [BsonId] ve [BsonRepresentation(BsonType.ObjectId)] nitelikleri eklenerek MongoDB uyumluluğu sağlanmıştır.
- **Veri Kayıt Mantığı:** DroneSimulationService içerisindeki her döngüde, üretilen veri SignalR ile ekrana gönderilirken eş zamanlı olarak _telemetryCollection.InsertOneAsync() metoduyla yerel MongoDB sunucusuna kalıcı olarak kaydedilmiştir. Bu sayede drone'ların geçmişe dönük rota ve batarya analizleri için veri güvenliği sağlanmıştır.

2. AWS Kinesis Veri Akış (Data Streaming) Mimarisi

Sistemin endüstriyel ölçekte binlerce drone'u destekleyebilmesi için "Büyük Veri" mimarisi kurgulanmış ve verilerin bulut ortamına aktarımı için **AWS Kinesis Data Streams** servisi seçilmiştir.

- **Kurulum ve Yetkilendirme:** AWS IAM üzerinden AmazonKinesisFullAccess yetkisine sahip bir kullanıcı oluşturulmuş ve güvenli bağlantı için Access Key / Secret Key çifti tanımlanmıştır.
- **Kinesis Client Entegrasyonu:** C# tarafında AmazonKinesisClient nesnesi oluşturulmuş ve bölge (Region) olarak veri merkezinin bulunduğu USEast1 (N. Virginia) atanmıştır.
- **PutRecord Operasyonu:** Her bir saniyelik veri paketi, JSON formatına serialize edilerek PutRecordRequest ile AWS bulutuna fırlatılmıştır.

3. Ölçeklenebilirlik ve "Sharding" Mantığı

Verilerin AWS tarafında nasıl işlendiğini ve sistemin neden dar boğaz (bottleneck) yapmadığını teknik olarak raporlamak amacıyla şu mimari detaylar uygulanmıştır:

- **Partition Key Kullanımı:** Veriler Kinesis'e gönderilirken PartitionKey parametresine her drone'un kendi DroneId bilgisi atanmıştır.
- **Veri Bütünlüğü (Ordering):** AWS bu anahtarı kullanarak, aynı drone'a ait tüm verilerin Kinesis içerisindeki aynı "Shard" (veri yolu) üzerinde toplanmasını sağlar. Bu sayede verilerin buluta ulaşma sırasının (Time-series) korunması garanti altına alınmıştır.
- **Monitoring:** AWS Console üzerindeki "Data Viewer" aracılığıyla, yerelden gönderilen verilerin gerçek zamanlı olarak bulut shard'larına ulaştığı ve JSON formatında işlendiği başarıyla doğrulanmıştır.

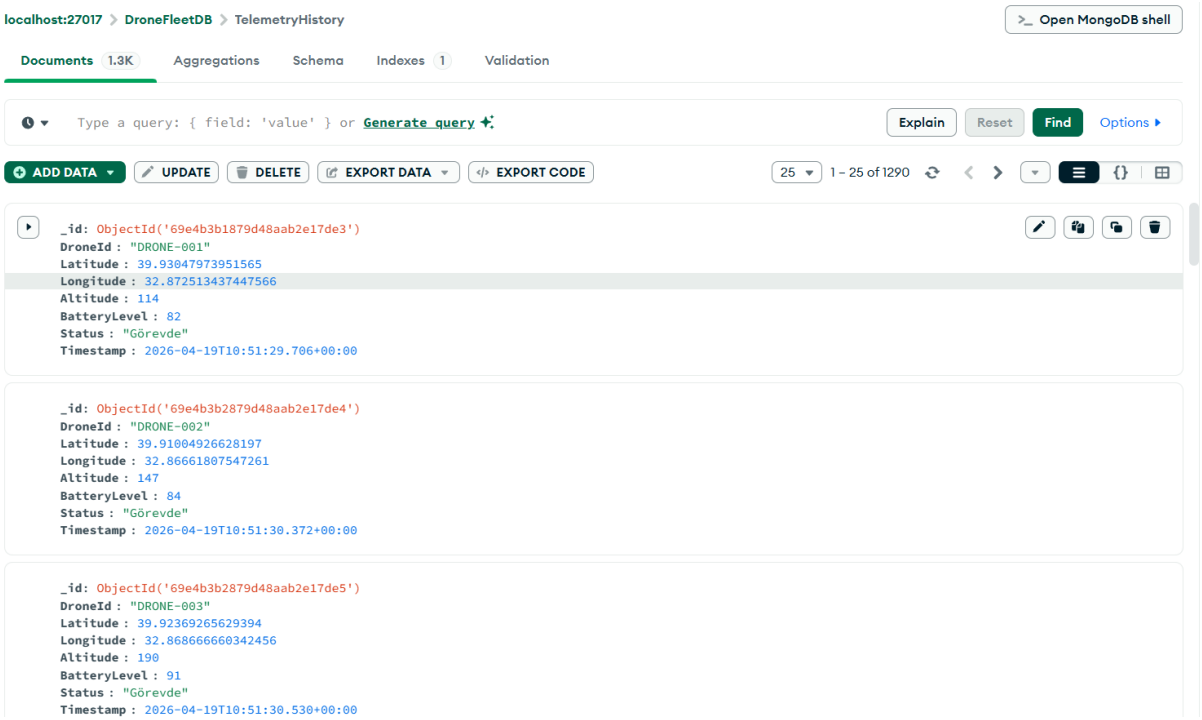
4. Güvenlik ve Sistem Kapatma Protokolü

Bulut kaynaklarının verimli kullanımı ve siber güvenlik risklerini minimize etmek amacıyla proje bitiminde şu protokoller uygulanmıştır:

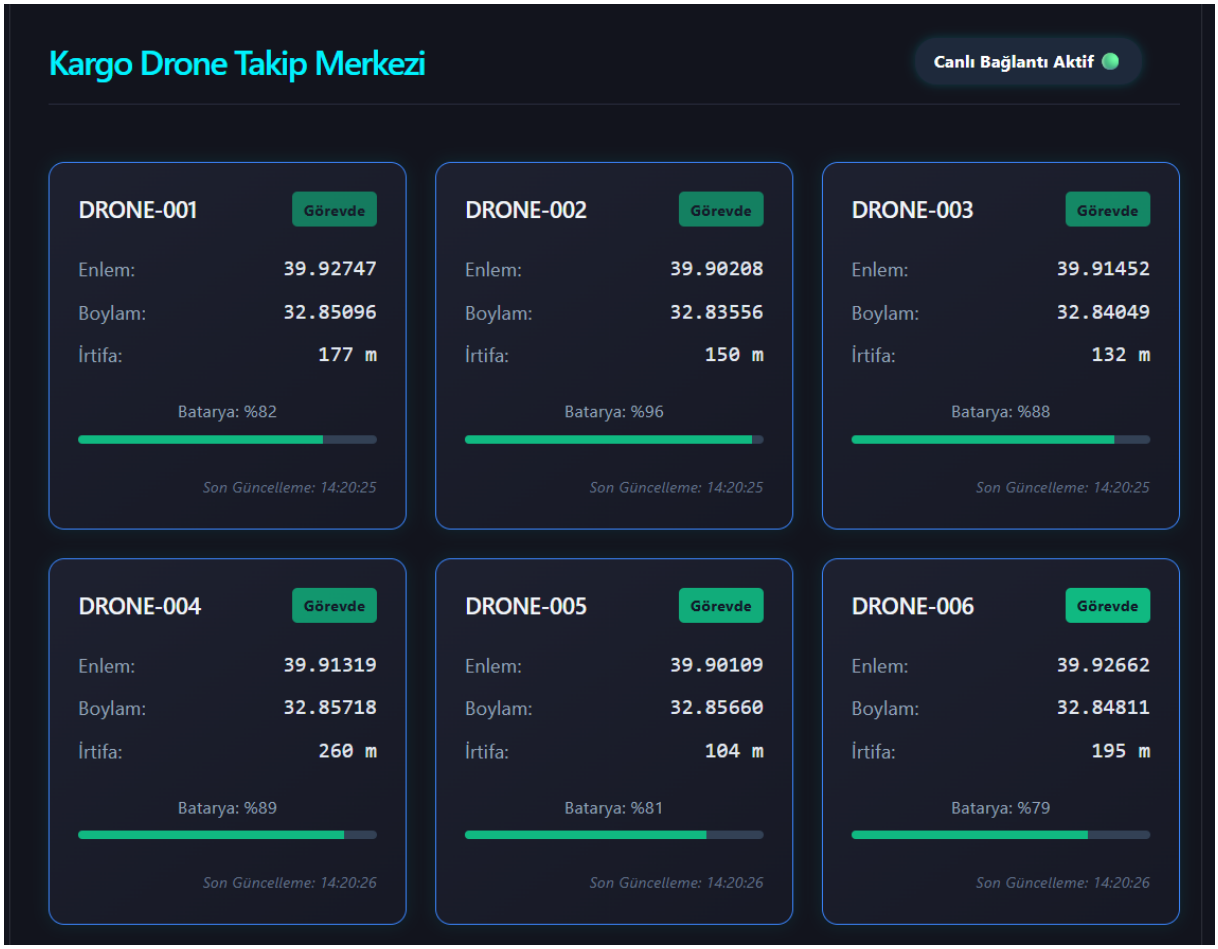
- **Credential Management:** Kaynak kodların GitHub gibi açık platformlarda paylaşılması durumunda güvenlik ihlali yaşanmaması için AWS anahtarları kod içerisinden temizlenmiş ve konfigürasyon dosyalarına (environment variables) taşınmıştır.

- **Resource Cleanup:** Test süreçlerinin ardından AWS Kinesis Stream hatları ve IAM yetkileri, gereksiz maliyet ve güvenlik açığı oluşmaması adına pasif hale getirilmiştir.

Bu proje kapsamında; **C# .NET 8** ile yüksek performanslı bir arka plan servisi, **SignalR** ile kesintisiz gerçek zamanlı iletişim, **React** ile modern bir kullanıcı arayüzü, **MongoDB** ile NoSQL veri depolama ve **AWS Kinesis** ile bulut tabanlı büyük veri mimarisi başarılı bir şekilde entegre edilmiştir. Sistem, uçtan uca bir IoT takip mekanizmasının tüm gereksinimlerini profesyonel standartlarda karşılamaktadır.



C# arka plan servisinden (BackgroundService) asenkron olarak aktarılan kargo drone telemetri verilerinin (Enlem, Boylam, Batarya Seviyesi vb.) JSON dokümanları halinde kalıcı olarak depolandığı görülmektedir. Sistemin otomatik olarak atadığı benzersiz _id parametreleri veri bütünlüğünü sağlamaktadır.

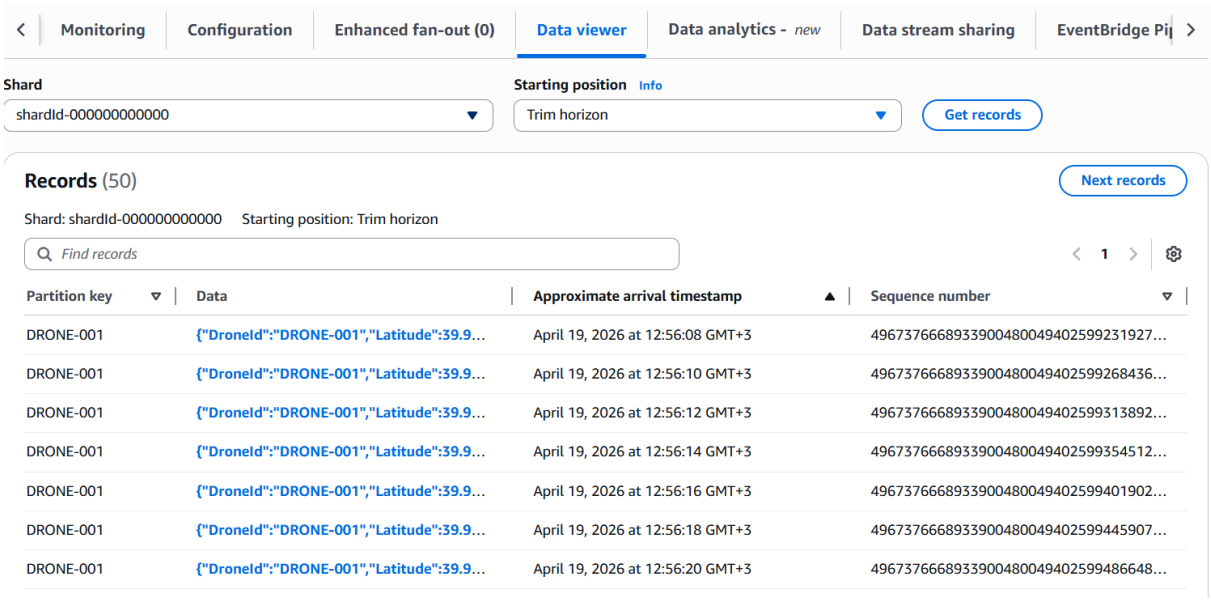


React ve CSS Grid mimarisi kullanılarak tasarlanan Gerçek Zamanlı Endüstriyel Kontrol Paneli (Dashboard). Arayüz, SignalR WebSocket bağlantısı üzerinden milisaniyeler içerisinde gelen telemetri verilerini işleyerek; drone'ların güncel koordinatlarını, durumlarını ve dinamik batarya göstergelerini (Progress Bar) son kullanıcıya anlık olarak yansıtmaktadır.

```
C:\Users\Berkay\source\repos

      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
info: DroneTrackingAPI.Services.DroneSimulationService[0]
      [SİSTEM AKTİF] Veriler SignalR, MongoDB ve AWS'ye işlendi.
```

ASP.NET Core üzerinde çalışan DroneSimulationService arka plan motorunun (Background Worker) terminal işlem logları. Simülasyon motoru 7/24 kesintisiz çalışarak gerçekçi uçuş rotaları ve batarya tüketim verileri üretmekte; bu verileri eş zamanlı olarak istemciye, yerel veritabanına ve bulut sunucularına iletmektedir.



AWS Kinesis Data Streams platformu üzerinde "Data Viewer" aracı ile anlık veri akışının izlenmesi. Yerel C# sunucusundan fırlatılan büyük veri (Big Data) paketleri, bulut ortamında başarılı bir şekilde yakalanmaktadır. Görselde, Kinesis'in yük dengelemesi (Load Balancing) ve veri sıralamasını korumak amacıyla PartitionKey (DroneId) parametresine göre ayırdığı Shard (Veri Yolu) yapısı net bir şekilde görülmektedir.

```
namespace DroneTrackingAPI.Models
{
    4 başvuru
    public class DroneTelemetry
    {
        // Her drone'un benzersiz kimliği (Örn: DRONE-001)
        2 başvuru
        public string DroneId { get; set; }

        // Konum verileri
        2 başvuru
        public double Latitude { get; set; }  // Enlem
        2 başvuru
        public double Longitude { get; set; } // Boylam
        2 başvuru
        public double Altitude { get; set; }  // İrtifa (Yükseklik)

        // Drone'un durumu
        5 başvuru
        public int BatteryLevel { get; set; } // % Batarya
        3 başvuru
        public string Status { get; set; }    // "Uçuyor", "Teslim Ediyor", "Dönüyor"

        // Verinin üretildiği an
        2 başvuru
        public DateTime Timestamp { get; set; } = DateTime.UtcNow;
    }
```

Bu kod bloğu, IoT cihazlarından (Drone) gelen ham telemetri verilerinin sistemde hangi tipte tutulacağını belirleyen "Entity" yapısıdır. NoSQL veritabanı (MongoDB) entegrasyonu için eklenen [BsonId] etiketi, veritabanına kaydedilen her verinin benzersiz bir kimlik (Object ID) kazanmasını sağlayarak veri bütünlüğünü korur.

```
using Microsoft.AspNetCore.SignalR;

namespace DroneTrackingAPI.Hubs
{
    // Hub sınıfından miras alarak buranın bir SignalR merkezi olduğunu belirtiyoruz.
    3 başvuru
    public class DroneHub : Hub
    {
        // İstemciler (Dashboard), bu sınıf üzerinden açılan kanal üzerinden
        // sunucuyla çift yönlü (Bidirectional) bağlantı kurar.
    }
}
```

SignalR kütüphanesinden miras alan bu sınıf, sistemin "Real-Time" (Gerçek Zamanlı) çalışma yeteneğini sağlar. Geleneksel HTTP istek-cevap mimarisinin aksine, sunucunun istemciye "Push" (Veri İtme) yapabilmesine olanak tanır ve sayfa yenilenmeden verilerin akmasını sağlar.

```
useEffect(() => {
  // DİKKAT: C# projen çalıştığında terminalde yazan port numarasını buraya girmelisin!
  const backendUrl = "http://localhost:5025/droneHub"; // KENDİ PORTUNLA DEĞİŞTİR

  const newConnection = new HubConnectionBuilder()
    .withUrl(backendUrl)
    .withAutomaticReconnect()
    .build();

  newConnection.start()
    .then(() => {
      setConnectionStatus('Canlı Bağlantı Aktif 🟢');

      // C#'tan fırlattığımız "ReceiveDroneData" isimli yayını dinliyoruz
      newConnection.on("ReceiveDroneData", (drone) => {
        setDrones(prevDrones => ({
          ...prevDrones,
          [drone.droneId]: drone // Gelen drone verisini ID'sine göre günceller
        }));
      });
    })
```

React tarafında kullanılan useEffect kancası (hook), sayfa açıldığında WebSocket bağlantısını kurar. connection.on metodu, sunucudan gelen her paketi yakalar ve "State Management" (Durum Yönetimi) kullanarak arayüzdeki sadece ilgili drone kartının (batarya barı, koordinat metni) güncellenmesini sağlar.

```
_kinesisClient = new AmazonKinesisClient(awsAccessKey, awsSecretKey, Amazon.RegionEndpoint.USEast1);

// MONGODB BAĞLANTISI (Yerel veya Bulut fark etmez, standart bağlantı kodudur)
var mongoClient = new MongoClient("mongodb://localhost:27017"); // Bilgisayarındaki yerel MongoDB
var database = mongoClient.GetDatabase("DroneFleetDB"); // Veritabanı adı
_telemetryCollection = database.GetCollection<DroneTelemetry>("TelemetryHistory"); // Tablo adı
```

Bu kod bloğu, DroneSimulationService sınıfının kurucu (constructor) metodudur. Servis başlatıldığında iki temel bağlantı kurulur: İlk olarak, sağlanan güvenlik anahtarları ve bölge (USEast1) bilgisiyle bulut tabanlı büyük veri akışını yönetecek olan **AWS Kinesis Client** yapılandırılır. İkinci olarak, yerel makinede (localhost:27017) çalışan **MongoDB** sunucusuna bağlanılarak, verilerin kalıcı olarak arşivleneceği DroneFleetDB veritabanı ve TelemetryHistory koleksiyonu (tablosu) sisteme tanıtılır.

```dotnet
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    _logger.LogInformation("AWS Kinesis ve MongoDB Bağlantıları Hazır! Filo havalanıyor...");
    Random rnd = new Random();

    var droneFleet = new List<DroneTelemetry>();
    for (int i = 1; i <= 6; i++)
    {
        droneFleet.Add(new DroneTelemetry
        {
            DroneId = $"DRONE-00{i}",
            Latitude = 39.920770 + (rnd.NextDouble() * 0.04 - 0.02),
            Longitude = 32.854110 + (rnd.NextDouble() * 0.04 - 0.02),
            Altitude = rnd.Next(100, 300),
            BatteryLevel = rnd.Next(80, 100),
            Status = "Görevde",
            Timestamp = DateTime.UtcNow
        });
    }
```

Arka plan motorunun ana çalışma alanı olan ExecuteAsync metodunun başlangıcıdır. Sonsuz döngüye girmeden önce, sistemde takip edilecek 6 adet kargo drone'unun ilk durumları (State) bir liste (droneFleet) içerisinde oluşturulur. Her bir drone'a benzersiz bir kimlik (DroneId), Ankara merkezli rastgele başlangıç koordinatları, gerçekçi bir uçuş irtifası (100-300m) ve dolu seviyelerde (%80-100) bir batarya kapasitesi atanarak simülasyonun tutarlılığı sağlanır.

```
// 1. REACT EKRANINA GÖNDER (Canlı Yayın)
await _hubContext.Clients.All.SendAsync("ReceiveDroneData", drone, stoppingToken);

// 2. MONGODB'YE KAYDET (Kalıcı Veritabanı - PDF Şartı)
try
{
    await _telemetryCollection.InsertOneAsync(drone, cancellationToken: stoppingToken);
}
catch (Exception ex)
{
    _logger.LogWarning($"MongoDB'ye yazılamadı (Kurulu olmayabilir): {ex.Message}");
}

// 3. AWS KINESIS'E GÖNDER (Büyük Veri Bulutu)
try
{
    var jsonData = JsonSerializer.Serialize(drone);
    var dataBytes = Encoding.UTF8.GetBytes(jsonData);
    var putRequest = new PutRecordRequest
    {
        StreamName = StreamName,
        Data = new MemoryStream(dataBytes),
        PartitionKey = drone.DroneId
    };
    await _kinesisClient.PutRecordAsync(putRequest, stoppingToken);
```

SignalR (Canlı Yayın): SendAsync metodu ile veriler, gecikmesiz olarak React arayüzüne (Frontend) itilir (Push).

MongoDB (Kalıcı Depolama): InsertOneAsync metodu ile veriler, geçmişe dönük analizler için NoSQL veritabanına fiziksel olarak yazılır.

AWS Kinesis (Büyük Veri Bulutu): Telemetri objesi JSON formatına dönüştürülüp (Serialize), PutRecordAsync metodu ile AWS bulutuna fırlatılır. Try-Catch blokları, dış servislerde yaşanabilecek olası kesintilerin ana simülasyon motorunu çökertmesini engellemek için güvenlik sübabı olarak kullanılmıştır.